
Data Structures

BS (CS) _Fall_2024

Lab_13 Tasks



Learning Objectives:

1. B Tree

Lab Tasks

Submission Instructions

1. Create a separate **.h** and **.cpp** file for each class.
2. Now create a new folder with name *ROLLNO_SEC_LAB13* e.g. **i23XXXX_A_LAB07**(Also name your **cpp** file like this.)
3. Move all of your files to this newly created directory and compress it into a **.zip file**. Return the test file as well.
4. Now you have to submit this zipped file on Google Classroom.
5. If you don't follow the above-mentioned submission instruction, you will be marked **zero**.
6. Plagiarism or AI generated code in the Lab Task will result in **zero** marks in the whole category.

Question 1

Problem Statement: Pascal's Triangle Using a B-Tree

Write a C++ program that uses a **B-Tree** to store and generate Pascal's Triangle. Each level of Pascal's Triangle should correspond to a level of the B-Tree. Implement the following functionalities:

1. Generate Pascal's Triangle:

Create a function to generate Pascal's Triangle up to a specified number of rows (n). Use the recursive formula:

$$C(n, k) = \frac{n!}{k! \times (n-k)!}$$

or by summing the two numbers above it in the triangle:

$$C(n, k) = C(n-1, k-1) + C(n-1, k)$$

2. Store Values in a B-Tree:

- Insert each row of Pascal's Triangle into the B-Tree, maintaining the properties of the B-Tree:
 - All keys in a node are sorted.
 - All keys in the left subtree are smaller than the smallest key in the node.
 - All keys in the right subtree are greater than the largest key in the node.

3. Display Pascal's Triangle:

- Perform a **level-order traversal** of the B-Tree to display Pascal's Triangle row by row.

4. Search Functionality:

- Implement a search function to check if a specific value exists in the Pascal's Triangle stored in the B-Tree.

Constraints and Requirements

1. Input:
 - The user provides the number of rows (n) to generate the Pascal's Triangle.
 - The user can input a value to search within the triangle.
2. Output:
 - Display the Pascal's Triangle using the B-Tree.
 - Indicate whether the searched value is present or not with useful info.

3. B-Tree Properties:

- Let the B-Tree have a minimum degree t (defined by the user or set as a constant). Ensure that the tree structure maintains balance during insertions.
- Implement standard B-Tree insertion logic for each row.

Question 3

Given the root of a B tree, imagine yourself standing on the **right side** of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1, [2, 3, 4], [5], [6, 7]]

Output: [1,4,7]

Explanation:

- At depth 0, the visible node is 1.
- At depth 1, the rightmost visible node is 4.
- At depth 2, the rightmost visible node is 7.

Example 2:

Input: root = [1,[2],[3]]

Output: [1,3]

Explanation:

- At depth 0, the visible node is 1.
- At depth 1, the rightmost visible node is 3.

Example 3:

Input: root = []

Output: []

Explanation:

There is no tree, so the output is an empty list.